

系统开发工具基础 · 实验报告（第 2 周）

姓名：甘如嫣 学号：25090032014

2026 年 8 月 31 日（周一，第 5–8 节）

授课内容：命令行环境；开发环境与工具；Debugging and Profiling。

1 第 5 题 控制一个可清理的后台任务

主题：命令行环境：进程、信号与任务控制 建议用时：12–15 分钟

实验目的：掌握前台/后台任务切换、挂起（Ctrl-Z）、jobs/bg 任务控制，以及用信号优雅终止可清理的任务。

实验步骤：将给定脚本保存为 q05/worker.sh 后执行：

```
1 cd q05
2 chmod +x worker.sh
3 ./worker.sh > stdout.log 2> stderr.log # 前台启动，stdout/stderr 分开重定向
4 # 按 Ctrl-Z 挂起前台任务
5 bg # 放入后台继续运行
6 jobs # 确认状态：[1]+ Running ...
7 PID=$(jobs -p) # 取得 PID（不手抄）
8 kill -TERM "$PID" # 发送 SIGTERM
9 wait "$PID" 2>/dev/null # 等待任务结束
10 cat cleanup.log # 应出现 CLEAN_EXIT
```

实验结果：jobs 显示任务处于 Running 状态；发送 SIGTERM 后任务退出，cleanup.log 中出现 CLEAN_EXIT；全程未使用 kill -9。

说明：脚本顶部的 trap '...' TERM INT 捕获终止信号，先写入 cleanup.log 再 exit 0，实现优雅退出。jobs -p 直接返回任务 PID，避免手工抄写带来的误差。

2 第 6 题 语义重构与本地开发反馈

主题：开发环境与工具 建议用时：12–15 分钟

实验目的：使用编辑器的 Python 语言服务器完成跳转、查找引用与符号重命名，并用 ruff 与 pytest 获得本地反馈。

实验步骤：

1. 在 q06 中按题目创建 math_utils.py、app.py、test_math_utils.py 三个文件。
2. 在课程环境中打开 q06，启用 Python 语言服务器。

3. 对 `total_price` 分别执行**跳转到定义与查找引用**。
4. 使用**重命名符号**把 `total_price` 改为 `calculate_total`，定义与两处引用同步更新。
5. 在 `app.py` 中临时加入未使用的 `import os`，用 `ruff` 检测并删除该行。
6. 运行 `ruff check` 与 `pytest`，保证全部通过。

重命名后的代码：

```
1 # math_utils.py
2 def calculate_total(price: float, count: int) -> float:
3     return price * count
```

```
1 # app.py
2 from math_utils import calculate_total
3 print(calculate_total(12.5, 4))
```

```
1 # test_math_utils.py
2 from math_utils import calculate_total
3 def test_total_price():
4     assert calculate_total(12.5, 4) == 50.0
```

实验结果： `ruff check` 无告警；`pytest` 输出 1 passed。

说明： 符号重命名是语义级重构，由语言服务器统一改写定义与所有引用，避免手工遗漏。未使用的导入由 `ruff`（基于 AST 的 linter）发现并删除，保持代码整洁。

3 第 7 题 用调试器定位归并排序缺陷

主题： Debugging **建议用时：** 12–15 分钟

实验目的： 使用调试器（pdb）定位并修复归并排序中的下标缺陷，并补充测试保证正确性。

实验步骤：

1. 将缺陷代码保存为 `q07/merge_sort.py` 并运行，输出 `[1, 5, 4, 3, 4, 5, 6, 9]`，与正确的 `[1, 1, 2, 3, 4, 5, 6, 9]` 不符。
2. `python -m pdb merge_sort.py`，执行 `break merge` 在 `merge` 函数设断点，用 `next` 单步，`p i`、`p j`、`p left[i]`、`p right[j]` 观察变量。
3. 定位缺陷：`else` 分支误用 `right[i]`，应为 `right[j]`。
4. 完成最小修复（不替换为 `sorted`）。

缺陷位置与修复：

```
1 while i < len(left) and j < len(right):
2     if left[i] <= right[j]:
3         result.append(left[i]); i += 1
4     else:
5         result.append(right[j]); j += 1    # 原为 right[i]，已修复
```

补充测试：

```
1 # test_merge_sort.py
2 from merge_sort import merge_sort
3 def test_given():
4     assert merge_sort([3,1,4,1,5,9,2,6]) == [1,1,2,3,4,5,6,9]
5 def test_duplicates():
6     assert merge_sort([4,2,4,2,1,1]) == [1,1,2,2,4,4]
```

实验结果：pytest 输出 2 passed；未使用 sorted 替换归并排序。

说明：else 分支对应“取右侧较小元素”，其下标应为右侧指针 j；误用 i 既可能越界，也会产生错误结果。

4 第 8 题 先测量，再优化慢速词频程序

主题：Profiling 建议用时：12-15 分钟

实验目的：用 time 与 cProfile 先测量、定位瓶颈，再对故意低效的去重程序做优化，并量化加速效果。

实验步骤：

```
1 cd q08
2 python generate_words.py           # 生成 words.txt (可重复)
3 time python wordfreq.py           # 运行 2 次并记录耗时
4 time python wordfreq.py
5 python -m cProfile -s cumulative wordfreq.py # 定位耗时函数/行
```

定位结果：主要耗时集中在 if word not in unique——在列表 unique 上线性查找，整体复杂度为 $O(n^2)$ 。

优化（用集合做 $O(1)$ 判重）：

```
1 words = open("words.txt", encoding="utf-8").read().split()
2 unique = set()
3 for word in words:
4     unique.add(word)
5 print("count=", len(unique))
```

实验结果：优化前后 count 均为 1000（未写死）；用相同输入再运行 2 次，分别取耗时中位数，计算加速比（优化前中位数 ÷ 优化后中位数，通常在十倍以上）。

说明：cProfile -s cumulative 按累计耗时排序，直接暴露热点。优化以集合替换列表去重，保持最终 count 不变，符合“先测量、再优化”的方法。